The actual text field is now surrounded by a normal Rails form_with form that submits a GET request to concerts_url, which Rails will interpret as ConcertsController with an index action—exactly the controller action for search that we just wrote.

The form has three data attributes, all of which are of interest to the Hotwire world. We declare data-turbo-frame as search-results, meaning that when the form is submitted, the response HTML should have a search-results frame that replaces the existing search-results frame, which you can see is at the end of the partial. We declare data-search-target, marking this element as the form target of the search controller. And we declare a Stimulus data-action so that when the form receives an input event, the submit method of the search controller is invoked.

The text field itself only changes to declare itself the input target of the search controller, and as mentioned, we end the snippet with an empty Turbo Frame named search-results.

Now all we need is the Stimulus controller to manage this. We haven't put a Submit button on the page—instead, we want the form to submit and display results whenever anything is typed in the text field.

Our first pass at the search controller is quite short—remember we need to run rails stimulus:manifest:update for this to be available.

chapter_08/01/app/javascript/controllers/search_controller.ts

```
import { Controller } from "@hotwired/stimulus"
import "form-request-submit-polyfill"

export default class SearchController extends Controller {
  static targets = ["form", "input"]
  formTarget: HTMLFormElement

  submit(): void {
    this.formTarget.requestSubmit()
  }
}
```

What happens in this code right now is that any typing in the text field triggers a